

Bitcoin Volatility Prediction

Presentation Script — Slide by Slide

Mohammed Hejairi | DSB FT2 Bahrain 2026 | Capstone Project

Slide 1 — Title

WHAT TO SAY

Good morning/afternoon. I'm Mohammed Hejairi, and my capstone project forecasts Bitcoin's realized volatility two days ahead, using only historical price and volume data.

Slide 2 — How Bitcoin's Price Swings

WHAT TO SAY

Before getting into the model, it's worth seeing why this matters. Bitcoin's price has been through massive swings since 2017 - over 80% crashes, twice, each followed by a rally to new all-time highs.

WHY WE DID THIS

I start here because the entire project is about measuring these swings, not predicting price direction. Volatility is the size of the swing, not which way it goes.

Slide 3 — What Is Volatility, and Why Predict It?

WHAT TO SAY

Volatility measures how much price moves, independent of direction. My analogy: predicting price is like guessing exactly where a boat will be tomorrow - nearly impossible. Predicting volatility is like guessing how rough the sea will be - a more realistic and still very useful question.

WHY WE DID THIS

Options traders, risk managers, and exchanges all need this number for concrete reasons: position sizing, Value-at-Risk, and margin requirements all depend on volatility, not on knowing tomorrow's price.

Slide 4 — Problem Statement

WHAT TO SAY

My aim is to predict Bitcoin's realized volatility over the next 2 days, 48 hours ahead of the features used to predict it - never the next hour's value.

WHY WE DID THIS

This is framed as a regression problem, not classification, because every downstream use - option pricing, VaR, margin decisions - needs a continuous number, not a high/moderate/low label.

Slide 5 — Data Overview & Feature Dictionary

WHAT TO SAY

The dataset is 79,000+ hourly BTC/USDT records spanning 2017 to 2026. Features fall into three groups: HAR-style realized-volatility windows, technical indicators, and one calendar feature.

WHY WE DID THIS

Hourly granularity was chosen specifically because it's fine enough to build a genuine 48-hour-ahead forecast without needing intraday tick data.

Slide 6 — Dataset & Data Cleaning

WHAT TO SAY

I fixed 29 timestamp gaps in the raw data, merged historical and live-collected data into one continuous file, and confirmed zero duplicate timestamps.

WHY WE DID THIS

Every feature in this project is a rolling-window calculation. If hours are silently missing, a '48-hour window' could actually span 55 real hours - corrupting the calculation with no obvious sign anything is wrong. Fixing gaps first was non-negotiable before any feature engineering.

Slide 7 — Feature Engineering: HAR Horizons & ATR_X

WHAT TO SAY

RV_day, RV_week, and RV_month measure the same volatility at three different lookback windows - 24 hours, 7 days, 30 days. ATR_X measures the true range price moved through each hour, scaled by price so it's comparable across any price level.

WHY WE DID THIS

The ACF plot showed volatility has long memory - it doesn't decay quickly - and the trend/seasonality decomposition showed the trend component dominates the series. Multiple lookback windows let the model sense which volatility regime it's currently in, something a single window can't do. ATR_X turned out to be the single strongest feature in the whole project, correlating at $r=0.685$ with the target.

Slide 8 — Feature Engineering: BB_width & What Got Dropped

WHAT TO SAY

Bollinger Band width measures how far price disperses from its own 20-hour moving average. I also engineered and tested three more candidates: Keltner Channel width, realized skewness, and the Relative Volatility Index.

WHY WE DID THIS

Not every idea works, and I wanted to show that honestly. Keltner Channel width correlated at 0.993 with ATR_X - essentially a duplicate - so I dropped it to avoid feeding the model the same signal twice. Skewness and RVI both came back with near-zero correlation with the target. Reporting what didn't work is part of a rigorous feature-selection process, not just what did.

Slide 9 — EDA: Temporal Features

WHAT TO SAY

The ACF decays slowly instead of dropping to zero quickly, and the ADF test confirms the series is stationary but has real memory - genuine volatility clustering, not random noise.

WHY WE DID THIS

This finding directly shaped how I split and cross-validated the data. Autocorrelated data breaks the random-shuffle assumption behind a normal train/test split or K-Fold CV - that's why every split in this project respects chronological order instead.

Slide 10 — Feature Selection: Correlation & Chi-Square

WHAT TO SAY

Every feature I kept correlates meaningfully with the target - none are statistical accidents. I also ran a formal chi-square test on calendar features: hour-of-day showed no relationship at all, but day-of-week did - a real weekly rhythm.

WHY WE DID THIS

I didn't want to just assume calendar effects exist or don't - I tested them formally instead of eyeballing a chart. That's what let me confidently drop hour-of-day and confidently add day-of-week as a real feature, backed by statistics rather than a guess.

Slide 11 — Using Time as a Feature: `dow_sin` & `dow_cos` (Illustration)

WHAT TO SAY

Raw day numbers confuse a model - Sunday and Monday look far apart numerically, even though they're neighbors on the calendar. The fix is to place each day on a circle using sine and cosine, so Sunday and Monday sit right next to each other, just like they really are.

WHY WE DID THIS

This is the standard technique for any cyclical variable. Since the chi-square test confirmed day-of-week is a real signal, encoding it properly - instead of feeding the model a misleading raw number 0 through 6 - is what let the model actually use that signal. It directly explains the R2 jump you'll see in Model 2.

Slide 12 — Using Time as a Feature: dow_sin & dow_cos (Detail)

WHAT TO SAY

In XGBoost specifically, an `add_time_features()` function builds all four cyclical columns - `hour_sin`, `hour_cos`, `dow_sin`, `dow_cos` - from the Date column. Day-of-week was kept as a real feature; hour-of-day ended up with almost zero importance, exactly as the chi-square test predicted.

WHY WE DID THIS

This slide is the detailed backup version of the previous one, in case a reader wants the exact implementation detail rather than just the visual intuition.

Slide 13 — Preprocessing

WHAT TO SAY

The target is built by shifting volatility 48 hours into the future. The data is split chronologically at a fixed date rather than randomly, and `TimeSeriesSplit` is used for every hyperparameter search.

WHY WE DID THIS

The horizon is deliberately set equal to the volatility window size - that guarantees the feature and the target share zero overlapping hours, so there is no leakage. And because the data is autocorrelated, a random split or standard K-Fold would let the model quietly peek at the future; chronological splitting is the only honest way to evaluate this fairly.

Slide 14 — Baseline: GARCH (Background)

WHAT TO SAY

GARCH is the industry-standard, most-cited model for volatility forecasting, going back to Engle's 1982 ARCH model and Bollerslev's 1986 extension.

WHY WE DID THIS

I used it as the baseline specifically because it sets the performance bar every other model in this project has to clear - if a model can't beat GARCH, it isn't adding value.

Slide 15 — Baseline: GARCH (Formula & Performance)

WHAT TO SAY

GARCH blends a floor level, the previous shock, and the previous variance estimate into a new forecast. On our data it scores $R^2 = -1.62$ - worse than simply guessing the test set's average every time.

WHY WE DID THIS

It performs badly here for a structural reason, not a bug: its forecast mean-reverts within roughly 34 hours, so by our 48-hour horizon it has already lost most of its ability to react to current conditions. It also never sees any of the engineered features, only raw returns.

Slide 16 — Model 1: EWMA (Background)

WHAT TO SAY

EWMA is RiskMetrics-style exponential smoothing, published by J.P. Morgan in 1996 to give risk desks a fast, transparent volatility estimate.

WHY WE DID THIS

It's a useful middle ground in complexity between GARCH and a full regression - one tunable parameter, no distributional assumptions, no feature engineering required.

Slide 17 — Model 1: EWMA (Formula & Performance)

WHAT TO SAY

The only parameter is lambda, controlling how much weight goes to history versus the newest data point. Tuned to lambda=0.99 via TimeSeriesSplit, it scores $R^2=0.140$ - comfortably clearing the GARCH baseline.

WHY WE DID THIS

Tuning toward a slower, smoother setting matched our 48-hour horizon far better than the fast-reacting default (which alone scored -0.125). But as a one-input model with no access to ATR_X or any other feature, it hits a hard ceiling below the required models that follow.

Slide 18 — Model 2: HAR-RV-X (Background)

WHAT TO SAY

HAR-RV-X is based on Corsi's 2009 Heterogeneous Autoregressive model, built specifically because realized volatility shows long memory across multiple timescales that a single-lag model discards.

WHY WE DID THIS

This is the natural fit for what the EDA already found: strong, slow-decaying autocorrelation and a dominant trend component. It's a regression on multiple realized-volatility windows plus the technical and calendar features.

Slide 19 — Model 2: HAR-RV-X (Formula & Performance)

WHAT TO SAY

Ridge-regularized regression on 7 features - the three RV windows, ATR_X, BB_width, and day-of-week. Tuned, it scores $R^2=0.310$ - the best of the three required models.

WHY WE DID THIS

Adding day-of-week, backed directly by the earlier chi-square result, was the single biggest jump in the entire project - R^2 went from 0.234 to 0.310 from that one change alone.

Slide 20 — Model 3: XGBoost (Background)

WHAT TO SAY

XGBoost is a gradient-boosted decision tree ensemble, introduced by Chen & Guestrin in 2016, dominant in applied machine learning for structured/tabular data.

WHY WE DID THIS

I included it to directly test whether the relationship between our features and volatility is genuinely nonlinear - something the linear HAR-RV-X structurally cannot capture.

Slide 21 — Model 3: XGBoost (Formula & Performance)

WHAT TO SAY

Untuned, XGBoost badly overfits at $R^2 = -0.14$. Tuned via GridSearchCV to shallow trees ($\text{max_depth}=3$) and a slow learning rate (0.05), it reaches $R^2=0.306$.

WHY WE DID THIS

This is the clearest overfitting story in the whole project - the before/after tuning contrast shows exactly why constraining model capacity matters on a small, noisy financial dataset.

Slide 22 — Model Comparison

WHAT TO SAY

This table lists every model I tried in the final notebook, including six extra attempts beyond the three required models. XGBoost with a log-transformed target ends up the single best result overall, at $R^2=0.324$.

WHY WE DID THIS

I wanted to be transparent about everything tested, not just the winners - including Facebook Prophet, which underperformed, and a detrending experiment that confirmed the model was already capturing that signal on its own through RV_month.

Slide 23 — Limitations

WHAT TO SAY

This model only ever sees price and volume history - no news, no regulation, no on-chain blockchain data. Even the strongest feature, ATR_X, only correlates at $r=0.685$ with the target.

WHY WE DID THIS

I include this because a moderate R^2 ceiling here is a predictable result of what information the model actually has access to, not a flaw in the modeling process itself.

Slide 24 — Operational Implications

WHAT TO SAY

This forecast is genuinely useful for position sizing, as an options-pricing input, and for risk reporting - but it should never be used as a sole trading signal or for real-time automated execution.

WHY WE DID THIS

An R^2 around 0.31 means the model explains roughly a third of the variance in future volatility. That makes it one legitimate input among several, not a crystal ball - and I wanted to be explicit about exactly where that line sits.

Slide 25 — Thank You

WHAT TO SAY

Thank you - I'm happy to take any questions.